IA e MNIST Apostila de estudo

Aulas 1 a 5 • Projeto MNIST-FPGA • Revisão de 09/10/2026

Fundamentos da representação de imagens e das redes neurais, com exemplos em Python: dos pixels à operação matricial de uma camada.

Código de referência: classe Network

Arquivo `modelo/modelo.py`, do repositório GilvanTWS/MNIST-FPGA. Esta é a versão consultada em 03/10/2026 e utilizada na primeira edição, preservada abaixo. Os comentários longos recebem apenas quebra visual de linha.

```
class Network(object):
    def __init__(self, sizes): #paramentro sizes contem o numero de neuronios das respectivas
    camadas. É um objeto do tipo lista
        self.num_layers = len(sizes)
        self.sizes = sizes
        self.biases = [np.random.randn(y,1) for y in sizes[1:]]
        self.weights = [np.random.randn(y,x) for x,y in zip(sizes[:-1], sizes[1:])]
#se quisermos um objeto Network com 2 neuronios na primeira camada, 3 na segunda e 1 na
    terceira (final).
#faremos: rede1 = Network([2,3,1])
#Bias e pesos sao inicializados aleatoreamente com o np.random.randn para gerar distribuições
    daussianas com 0 de media e desvio padrao 1
```

Para executar essa classe, é necessário acrescentar `import numpy as np` antes dela. O import não consta no arquivo original. A classe inicializa a estrutura, os biases e os pesos; esse trecho ainda não implementa treinamento.

Percurso de estudo

Etapa	Conteúdo
Aula 1	Imagem, pixels, MNIST, entradas e rótulos
Aula 2	Neurônio, pesos e soma ponderada
Aula 3	Bias e função de ativação
Aula 4	Camada, matriz de pesos e listas com for
Aula 5	NumPy, shape, np.dot e ligação com Network

Fonte do código: github.com/GilvanTWS/MNIST-FPGA • commit 44134214e3c93cae254e73671057c369285015bd. Conteúdo didático adaptado das Aulas 1 a 5 de “Contexto do projeto”.

Aula 1 - Imagens como números

1.1 Pixels e matrizes

O objetivo é construir um programa que receba a imagem de um algarismo escrito à mão e identifique qual número ela representa. A rede recebe valores numéricos que descrevem a imagem.

O MNIST reúne imagens dos algarismos de 0 a 9. Cada imagem tem **28 linhas e 28 colunas de pixels**. Um exemplo menor, de 5 × 5 pixels, permite visualizar a representação de um possível algarismo 1:

```
■ ■ □ ■ ■
■ □ □ ■ ■
■ ■ □ ■ ■
■ ■ □ ■ ■
■ □ □ □ ■
```

Adotando preto = 0 e branco = 1, o desenho corresponde à matriz:

```
0 0 1 0 0
0 1 1 0 0
0 0 1 0 0
0 0 1 0 0
0 1 1 1 0
```

Em Python, essa matriz pode ser representada por uma lista de listas. Cada lista interna corresponde a uma linha da imagem.

```
imagem = [
    [0, 0, 1, 0, 0],
    [0, 1, 1, 0, 0],
    [0, 0, 1, 0, 0],
    [0, 0, 1, 0, 0],
    [0, 1, 1, 1, 0]
]
```

1.2 Intensidade dos pixels

Uma imagem em tons de cinza não se limita a dois valores. Na representação de 8 bits, cada pixel pode assumir uma intensidade entre 0 e 255:

```
0     = preto
50    = cinza bem escuro
128   = cinza
220   = cinza claro
255   = branco
```

A associação entre intensidade e cor depende da convenção usada. O conceito central é que cada pixel possui um valor numérico. Uma região da imagem poderia ser:

```
imagem = [
    [0,   0,   30,  180, 255],
    [0,   20,  200, 255, 170],
    [0,   0,   210, 250, 30],
]
```

Aula 1 - Entrada, saída e rótulos

1.3 De uma matriz 28 × 28 para 784 entradas

28 × 28 = **784 pixels**

Ao organizar os pixels em uma sequência, a matriz torna-se um vetor com 784 elementos. Cada posição do vetor fornece uma entrada para a rede.

```
IMAGEM

28 × 28 pixels
      ↓
transformamos em números
      ↓
784 valores
      ↓
┌──────────────────┐
│  REDE NEURAL     │
└──────────────────┘
      ↓
   resposta
```

```
x1   → pixel 1
x2   → pixel 2
x3   → pixel 3
...
x784 → pixel 784
```

1.4 Classes e predição

Existem dez classes possíveis: os algarismos de 0 a 9. A rede pode produzir um valor para cada classe; a classe com maior valor é escolhida como predição. No exemplo, 7 tem o maior valor, 0,84:

```
classe:  0     1     2     3     4     5     6     7     8     9
valor:  0.01  0.02  0.01  0.03  0.01  0.04  0.01  0.84  0.02  0.01

predição = 7
```

1.5 Dados x e resposta correta y

x representa os dados de entrada. y representa a resposta correta, chamada de **label** ou **rótulo**. Para uma imagem do número 7, o rótulo é 7; para uma imagem do número 3, o rótulo é 3.

```
x = imagem_do_numero_7
y = 7

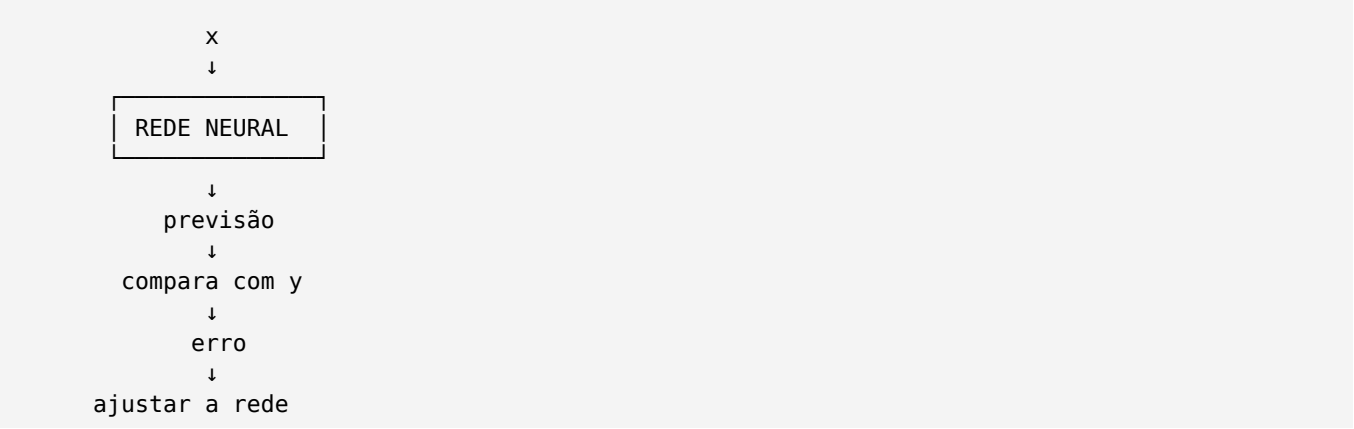
x = imagem_do_numero_3
y = 3
```

Os nomes de imagem acima são ilustrativos: representam os dados numéricos correspondentes a cada imagem. Um conjunto de treinamento contém muitos pares (x, y).

Aula 1 - Treinamento e primeiro programa

1.6 Ideia de treinamento

Se uma imagem representa o algarismo 1 e a rede prevê 7, há um erro. Treinar consiste em comparar a previsão com o rótulo correto e ajustar parâmetros para reduzir os erros. O processo é repetido em muitos exemplos.



Em vez de escrever uma regra manual para cada formato de algarismo, o treinamento ajusta parâmetros como pesos e biases. A regra matemática desse ajuste será estudada depois.

1.7 Prática: 01_imagem.py

```
imagem = [
    [0, 0, 1, 0, 0],
    [0, 1, 1, 0, 0],
    [0, 0, 1, 0, 0],
    [0, 0, 1, 0, 0],
    [0, 1, 1, 1, 0]
]

for linha in imagem:
    print(linha)
```

`imagem` é uma lista de cinco listas. `imagem[0]` acessa a primeira linha e `imagem[1]` acessa a segunda, pois os índices começam em zero. O laço `for linha in imagem` percorre as linhas, e `print(linha)` mostra cada uma.

```
python 01_imagem.py
```

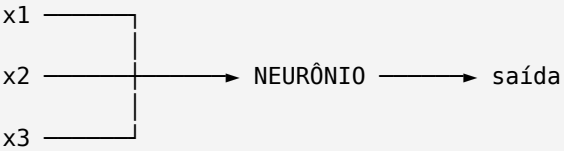
```
[0, 0, 1, 0, 0]
[0, 1, 1, 0, 0]
[0, 0, 1, 0, 0]
[0, 0, 1, 0, 0]
[0, 1, 1, 1, 0]
```

Exercício: execute o programa, identifique a primeira e a segunda linha e relacione os valores 0 e 1 ao desenho de pixels.

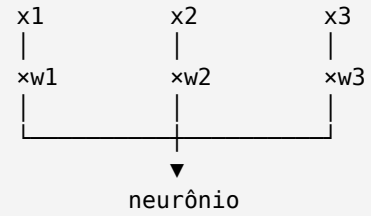
Aula 2 - O primeiro neurônio

2.1 Entradas e pesos

Um neurônio artificial recebe entradas numéricas. Neste exemplo, são três entradas: $x_1 = 1$, $x_2 = 0$ e $x_3 = 1$. Elas poderiam representar três pixels de uma imagem.



Cada entrada recebe um **peso** (weight), indicado por w. O peso controla quanto e de que maneira a entrada influencia o resultado.



2.2 Soma ponderada

Com pesos $w_1 = 0{,}5$, $w_2 = 0{,}2$ e $w_3 = -0{,}3$, multiplica-se cada entrada pelo peso correspondente e somam-se as contribuições:

$$\text{soma} = x_1 w_1 + x_2 w_2 + x_3 w_3$$
$$\text{soma} = (1)(0.5) + (0)(0.2) + (1)(-0.3)$$
$$\text{soma} = 0.5 + 0 - 0.3 = \mathbf{0.2}$$

Entrada	Peso	Contribuição
$x_1 = 1$	$w_1 = 0.5$	+0.5
$x_2 = 0$	$w_2 = 0.2$	0
$x_3 = 1$	$w_3 = -0.3$	-0.3

Para as entradas não negativas deste exemplo, o peso positivo aumenta a soma e o negativo a reduz. Uma entrada nula contribui com zero. Para a mesma entrada, aumentar a magnitude do peso aumenta a magnitude da contribuição.

2.3 Produto escalar

As entradas e os pesos também podem ser vistos como vetores. A soma das multiplicações entre posições correspondentes é o **produto escalar**:

```
x = [1, 0, 1]       w = [0.5, 0.2, -0.3]

x · w = (1)(0.5) + (0)(0.2) + (1)(-0.3) = 0.2
```

Aula 2 - Implementação e parâmetros

2.4 Prática: 02_neuronio.py, sem bias

```
x1 = 1
x2 = 0
x3 = 1

w1 = 0.5
w2 = 0.2
w3 = -0.3

soma = x1 * w1 + x2 * w2 + x3 * w3

print(soma)
```

O resultado esperado é aproximadamente 0,2. Um valor como `0.19999999999999998` pode aparecer devido à representação de números de ponto flutuante; isso não indica erro na fórmula.

2.5 Inclusão do bias

O bias é um parâmetro adicional somado à soma ponderada. Com b = 0,1, o resultado passa de 0,2 para 0,3:

$z = x_1w_1 + x_2w_2 + x_3w_3 + b = 0.2 + 0.1 = \mathbf{0.3}$

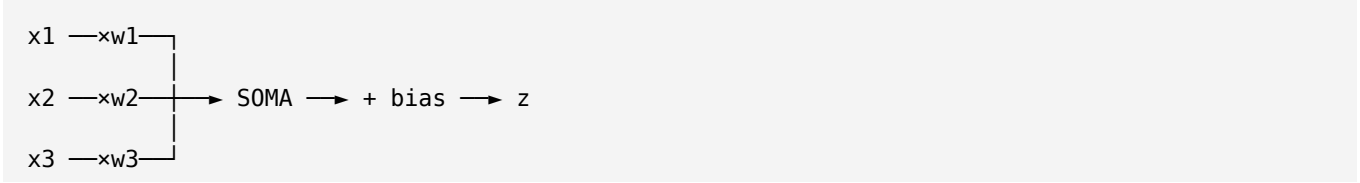
```
x1 = 1
x2 = 0
x3 = 1

w1 = 0.5
w2 = 0.2
w3 = -0.3

bias = 0.1

z = x1 * w1 + x2 * w2 + x3 * w3 + bias

print(z)
```



2.6 Origem dos pesos

Os valores usados nos exemplos foram escolhidos para facilitar as contas. Uma rede costuma iniciar seus pesos de forma aleatória. No código de referência, `np.random.randn(...)` faz essa inicialização. Durante o treinamento, pesos e biases são ajustados para diminuir o erro.

Exercício: execute as duas versões e compare os resultados sem bias e com `bias = 0.1`.

Aula 3 - O papel do bias

3.1 Um neurônio sem bias

Com uma entrada e peso w = 3, a expressão é z = 3x. Para x = 2, o resultado é 6:

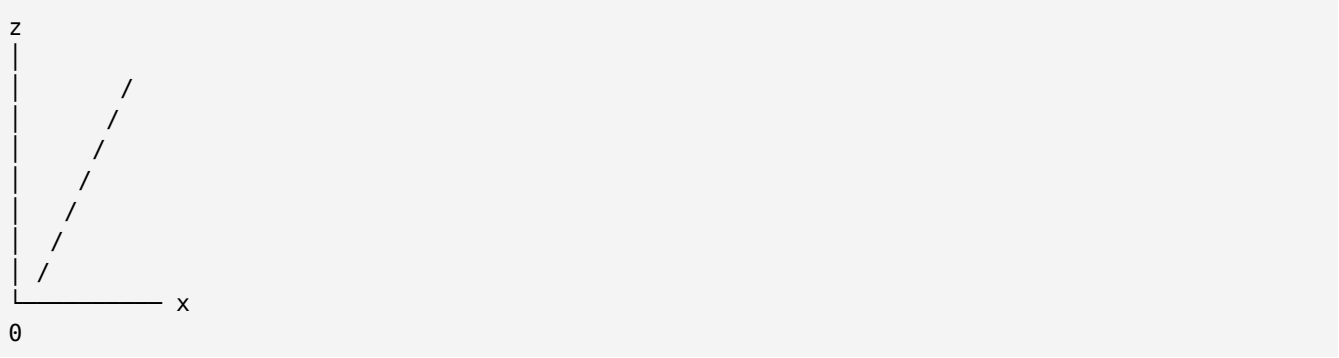
```
x = 2
w = 3

z = x * w

print(z)
```

```
x = 0 → z = 0
x = 1 → z = 3
x = 2 → z = 6
x = 3 → z = 9
```

A reta z = 3x passa pela origem. Sem bias, x = 0 implica z = 0, qualquer que seja o peso, pois 0 · w = 0.



3.2 A relação com y = ax + b

O bias permite deslocar a reta. Com w = 3 e b = 2, a expressão torna-se z = 3x + 2. Mesmo quando x = 0, a saída antes da ativação é 2:

```
x = 0
w = 3
bias = 2

z = x * w + bias

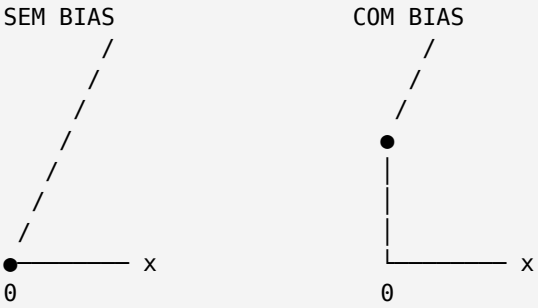
print(z)
```

Sem bias: z = 3x
Com bias: z = 3x + 2
Forma conhecida: y = ax + b

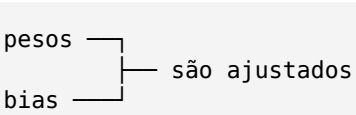
Nessa comparação, w corresponde à inclinação a, enquanto o bias corresponde ao termo independente b. O bias permite que a reta não passe pela origem.

Aula 3 - Da soma à ativação

3.3 Deslocamento e parâmetros treináveis



Os pesos controlam a influência das entradas; o bias ajusta o deslocamento da soma e, em conjunto com a ativação, o ponto em que o neurônio muda sua resposta. **O bias também é aprendido.**



3.4 Equação geral do neurônio

$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$

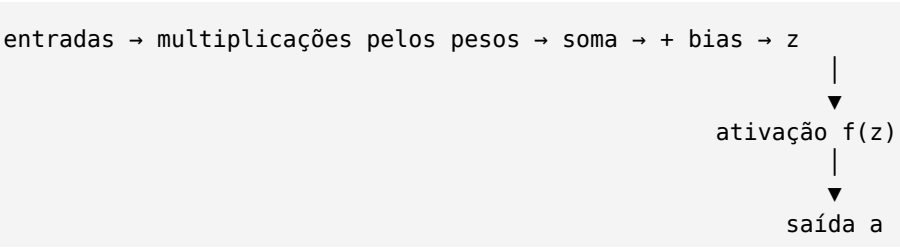
$$\mathbf{z = \sum_{i=1}^{n} w_i x_i + b}$$

A soma ∑ percorre todas as entradas, de i = 1 até n. Para um neurônio, a forma compacta é z = w · x + b. Com bias, essa transformação é tecnicamente afim; sem bias, é linear.

3.5 Função de ativação

Após calcular z, aplica-se uma função f para obter a saída a:

a = f(z)




```
z → resultado antes da ativação
a → resultado depois da ativação
```

A ativação define como o neurônio transforma a soma recebida. O valor z = 0,3, por exemplo, pode ser convertido em uma decisão binária por uma função degrau.

Aula 3 - Função degrau

3.6 Regra de decisão

A função degrau usada nesta apostila retorna 1 quando z é maior que zero e retorna 0 nos demais casos, inclusive z = 0.

```
         { 1, se z > 0
f(z) =   {
         { 0, se z ≤ 0
```

Exemplos: f(0,3) = 1 e f(-0,7) = 0. Em Python, essa regra é expressa com `if` e `else`.

3.7 Prática: 03_ativacao.py

```
x1 = 1
x2 = 0
x3 = 1

w1 = 0.5
w2 = 0.2
w3 = -0.3

bias = 0.1

z = x1 * w1 + x2 * w2 + x3 * w3 + bias

if z > 0:
    saida = 1
else:
    saida = 0

print("z:", z)
print("saida:", saida)
```

Resultado aproximado:

```
z: 0.3
saida: 1
```

3.8 Encapsulamento em uma função

A mesma regra pode ser definida uma vez e chamada pelo nome `degrau`:

```
def degrau(z):
    if z > 0:
        return 1
    else:
        return 0
```

```
saida = degrau(z)
```

Essa separação distingue o cálculo da soma ponderada e do bias da aplicação da ativação.

Aula 3 - Programa e experimento

3.9 Versão completa com função

```
def degrau(z):
    if z > 0:
        return 1
    else:
        return 0


x1 = 1
x2 = 0
x3 = 1

w1 = 0.5
w2 = 0.2
w3 = -0.3

bias = 0.1

z = x1 * w1 + x2 * w2 + x3 * w3 + bias

saida = degrau(z)

print("z:", z)
print("saida:", saida)
```

3.10 Efeito de alterar apenas o bias

Mantenha as entradas e os pesos e substitua `bias = 0.1` por `bias = -1`. A conta passa a ser:

z = (1)(0.5) + (0)(0.2) + (1)(-0.3) - 1
z = 0.5 + 0 - 0.3 - 1 = **-0.8**
degrau(-0.8) = **0**

Apenas a alteração do bias muda a decisão do neurônio, de 1 para 0. **Exercício:** calcule o resultado antes da execução e confirme-o com o programa.

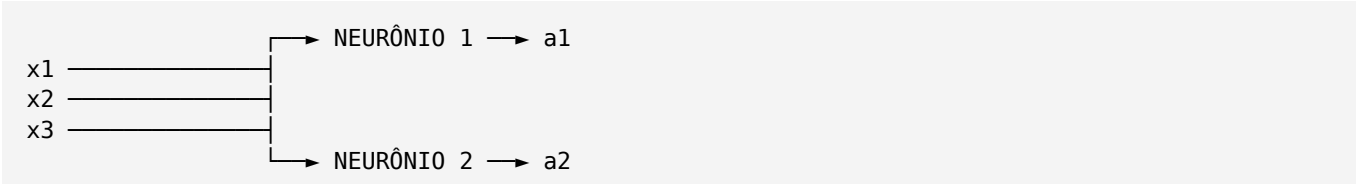
3.11 Limite didático do degrau

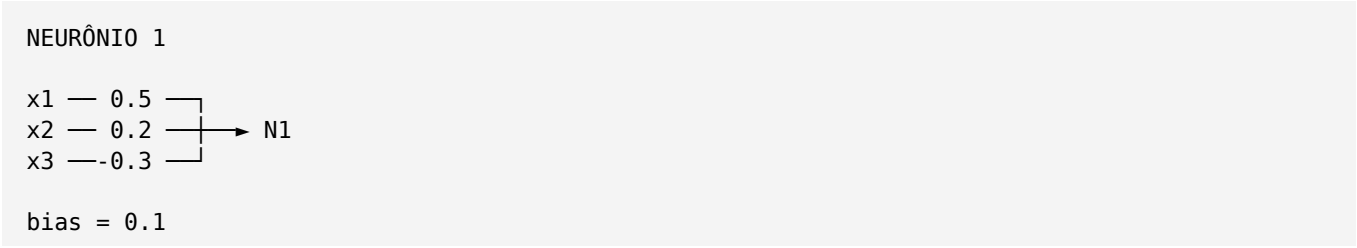
O degrau é útil para compreender uma decisão binária. No treinamento de redes neurais, outras ativações, como sigmoid, ReLU e softmax, têm diferentes finalidades. A escolha se relaciona às derivadas e ao método de treinamento; essas funções ainda não são desenvolvidas nas aulas reunidas aqui.

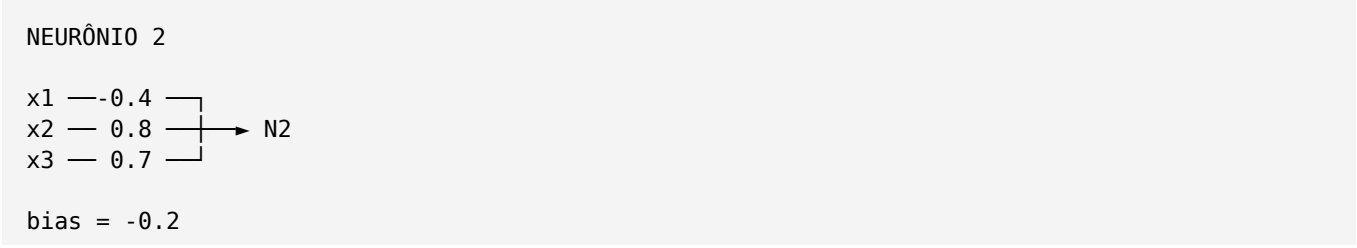
Aula 4 - De um neurônio a uma camada

4.1 Mesmas entradas, parâmetros diferentes

Uma camada pode reunir vários neurônios que recebem as mesmas entradas. Cada neurônio tem seus próprios pesos e seu próprio bias.





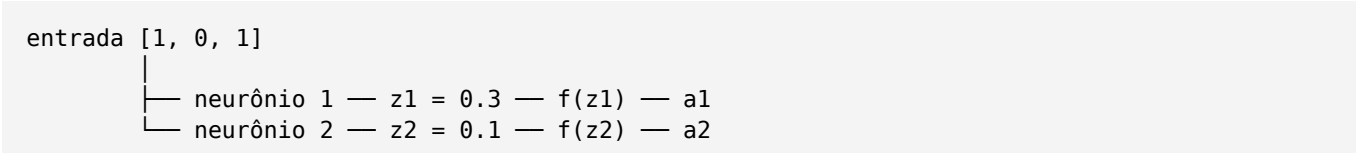


4.2 Cálculo dos dois neurônios

Para $x_1 = 1$, $x_2 = 0$ e $x_3 = 1$:

$z_1 = (1)(0.5) + (0)(0.2) + (1)(-0.3) + 0.1 = \mathbf{0.3}$
$z_2 = (1)(-0.4) + (0)(0.8) + (1)(0.7) - 0.2 = \mathbf{0.1}$

Entram três números e são produzidos dois valores antes da ativação. Se uma função de ativação for aplicada, as saídas serão $a_1 = f(z_1)$ e $a_2 = f(z_2)$.



Aula 4 - Implementação explícita

4.3 Prática: 04_camada.py

A primeira versão escreve cada multiplicação explicitamente. Os resultados esperados são aproximadamente 0,3 e 0,1.

```
x1 = 1
x2 = 0
x3 = 1

# Pesos do neurônio 1
w11 = 0.5
w12 = 0.2
w13 = -0.3

b1 = 0.1

# Pesos do neurônio 2
w21 = -0.4
w22 = 0.8
w23 = 0.7

b2 = -0.2

z1 = x1 * w11 + x2 * w12 + x3 * w13 + b1

z2 = x1 * w21 + x2 * w22 + x3 * w23 + b2

print("Neurônio 1:", z1)
print("Neurônio 2:", z2)
```

4.4 Significado dos índices dos pesos

Em w_{ij}, o primeiro índice identifica o neurônio de destino e o segundo identifica a entrada. Assim, w12 é o peso da entrada 2 para o neurônio 1.

```
w11 → peso da entrada 1 para o neurônio 1
w12 → peso da entrada 2 para o neurônio 1
w13 → peso da entrada 3 para o neurônio 1

w21 → peso da entrada 1 para o neurônio 2
w22 → peso da entrada 2 para o neurônio 2
w23 → peso da entrada 3 para o neurônio 2
```

4.5 Por que organizar os pesos?

Com 784 entradas e 30 neurônios, cada neurônio precisa de 784 pesos. O total é:

784 × 30 = **23.520 pesos**

Escrever uma variável e uma multiplicação para cada peso torna o programa difícil de manter. Vetores e matrizes organizam os mesmos valores sem alterar a matemática.

Aula 4 - Vetores e matriz de pesos

4.6 Vetor de entradas e vetores de pesos

As entradas podem ser armazenadas em `x = [1, 0, 1]`. Como Python conta a partir de zero, `x[0] = 1`, `x[1] = 0` e `x[2] = 1`. Cada neurônio também possui um vetor de pesos:

```
x  = [ 1,    0,    1   ]

w1 = [ 0.5,  0.2, -0.3 ]
w2 = [-0.4,  0.8,  0.7 ]
```

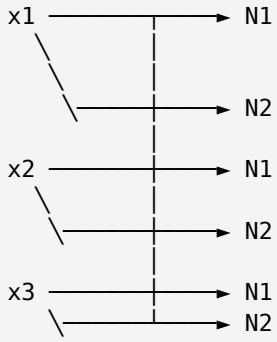
4.7 Matriz de pesos

Colocando um vetor de pesos em cada linha, obtém-se a matriz W:

```
    ┌                  ┐
W = │  0.5  0.2  -0.3 │   ← pesos do neurônio 1
    │ -0.4  0.8   0.7 │   ← pesos do neurônio 2
    └                  ┘

        ↑    ↑     ↑
       x1   x2    x3
```

A matriz tem **2 linhas e 3 colunas**: duas linhas para os dois neurônios, e três colunas para as três entradas. Cada entrada está conectada aos dois neurônios:



4.8 Regra das dimensões

Situação	Dimensão de W	Quantidade de pesos
n entradas e m neurônios	m × n	m · n
3 entradas e 2 neurônios	2 × 3	6
784 entradas e 30 neurônios	30 × 784	23.520

A ordem é **neurônios da camada de destino × entradas da camada anterior**. Essa organização será a mesma utilizada pela classe `Network`.

Aula 4 - Listas e laço for

4.9 Cálculo com listas

O programa abaixo calcula o primeiro neurônio percorrendo os pares entrada-peso. O bias é somado ao final.

```
entradas = [1, 0, 1]

pesos_n1 = [0.5, 0.2, -0.3]
pesos_n2 = [-0.4, 0.8, 0.7]

b1 = 0.1
b2 = -0.2

z1 = 0

for i in range(len(entradas)):
    z1 += entradas[i] * pesos_n1[i]

z1 += b1

print(z1)
```

4.10 Entendimento do laço

`len(entradas)` retorna 3. `range(3)` fornece os índices 0, 1 e 2. Em cada repetição, `entradas[i] * pesos_n1[i]` calcula uma contribuição. A instrução `z1 += valor` equivale a `z1 = z1 + valor`.

Etapa	Multiplicação	Acumulador z1
Inicialização	-	0
i = 0	1 × 0.5	0.5
i = 1	0 × 0.2	0.5
i = 2	1 × (-0.3)	0.2
Somar bias	+ 0.1	0.3

4.11 Exercício: segundo neurônio

Implemente outro laço usando `pesos_n2` e `b2` para calcular z_2. Compare com a implementação explícita. As duas versões devem produzir aproximadamente $z_1 = 0{,}3$ e $z_2 = 0{,}1$. Nesta etapa, use apenas listas e `for`.

O laço realiza a mesma soma ponderada: $x_1w_1 + x_2w_2 + \cdots + x_nw_n$. Essa sequência de multiplicações e somas também é relevante para a futura implementação em FPGA.

Aula 5 - NumPy e operações numéricas

5.1 Do laço à álgebra linear

O cálculo de um neurônio é um produto escalar seguido da soma do bias: **z = w · x + b**. Para x = [1, 0, 1] e w = [0.5, 0.2, -0.3], o produto escalar é 0,2; somando b = 0,1, obtém-se z = 0,3.

NumPy fornece arrays para representar vetores e matrizes e realizar operações numéricas. Uma lista comum de Python tem comportamento diferente de um array.

5.2 Multiplicação de lista

```
x = [1, 0, 1]

print(x * 2)
```

Resultado:

```
[1, 0, 1, 1, 0, 1]
```

Na lista, multiplicar por 2 repete os elementos. Não multiplica cada valor por 2.

5.3 Instalação e importação

Com o ambiente Python de estudo ativo, instale a biblioteca, se necessário:

```
pip install numpy
```

O comando `import numpy as np` importa NumPy com o apelido `np`. Assim, `numpy.array(...)` pode ser escrito como `np.array(...)`.

5.4 Primeiro array: 05_numpy.py

```
import numpy as np

entradas = np.array([1, 0, 1])

print(entradas)
```

A impressão é `[1 0 1]`. Ao executar a multiplicação abaixo, NumPy opera elemento por elemento:

```
print(entradas * 2)

# Resultado: [2 0 2]
```

[1, 0, 1] × 2 = [2, 0, 2]

Aula 5 - Formato dos arrays: shape

5.5 Array com uma dimensão

```
entradas = np.array([1, 0, 1])
print(entradas.shape)

# Resultado: (3,)
```

shape informa o tamanho de cada dimensão do array. `(3,)` significa um array de uma dimensão com três elementos. Ele não é uma matriz com três linhas e uma coluna; uma matriz desse tipo teria formato `(3, 1)`.

5.6 Matriz com duas dimensões

```
matriz = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

print(matriz)
print(matriz.shape)
```

Resultado:

```
[[1 2 3]
 [4 5 6]]

(2, 3)
```

A primeira dimensão conta as linhas; a segunda, as colunas. Portanto, `(2, 3)` significa duas linhas e três colunas.

5.7 Formato da matriz de pesos

```
pesos = np.array([
    [ 0.5, 0.2, -0.3],
    [-0.4, 0.8,  0.7]
])
```

```
print(pesos.shape)

# Resultado: (2, 3)
```

Cada linha corresponde a um neurônio, e cada coluna a uma entrada. A primeira linha contém [0.5, 0.2, -0.3]; a segunda, [-0.4, 0.8, 0.7].

Objeto	Formato	Significado
entradas	(3,)	3 entradas
pesos	(2, 3)	2 neurônios, 3 pesos por neurônio
biases nesta aula	(2,)	1 bias para cada neurônio

Aula 5 - Produto matricial com np.dot

5.8 Cálculo sem os biases

```
import numpy as np

entradas = np.array([1, 0, 1])

pesos = np.array([
    [ 0.5, 0.2, -0.3],
    [-0.4, 0.8,  0.7]
])

resultado = np.dot(pesos, entradas)

print(resultado)
```

Resultado aproximado:

```
[0.2 0.3]
```

NumPy calcula o produto escalar de cada linha de `pesos` com `entradas`:

Linha 1: (0.5)(1) + (0.2)(0) + (-0.3)(1) = **0.2**
Linha 2: (-0.4)(1) + (0.8)(0) + (0.7)(1) = **0.3**

```
     ⎡                  ⎤   ⎡   ⎤   ⎡     ⎤
Wx = ⎢  0.5  0.2  -0.3 ⎥ × ⎢ 1 ⎥ = ⎢ 0.2 ⎥
     ⎣ -0.4  0.8   0.7 ⎦   ⎢ 0 ⎥   ⎣ 0.3 ⎦
                           ⎣ 1 ⎦
```

O desenho mostra a notação matemática com vetor-coluna. No programa acima, `entradas` é um array 1D, com `shape` igual a `(3,)`. A operação corresponde às mesmas somas realizadas pelos laços da Aula 4.

5.9 Inclusão dos biases

```
biases = np.array([0.1, -0.2])
z = np.dot(pesos, entradas) + biases
print(z)

# Resultado aproximado: [0.3 0.1]
```

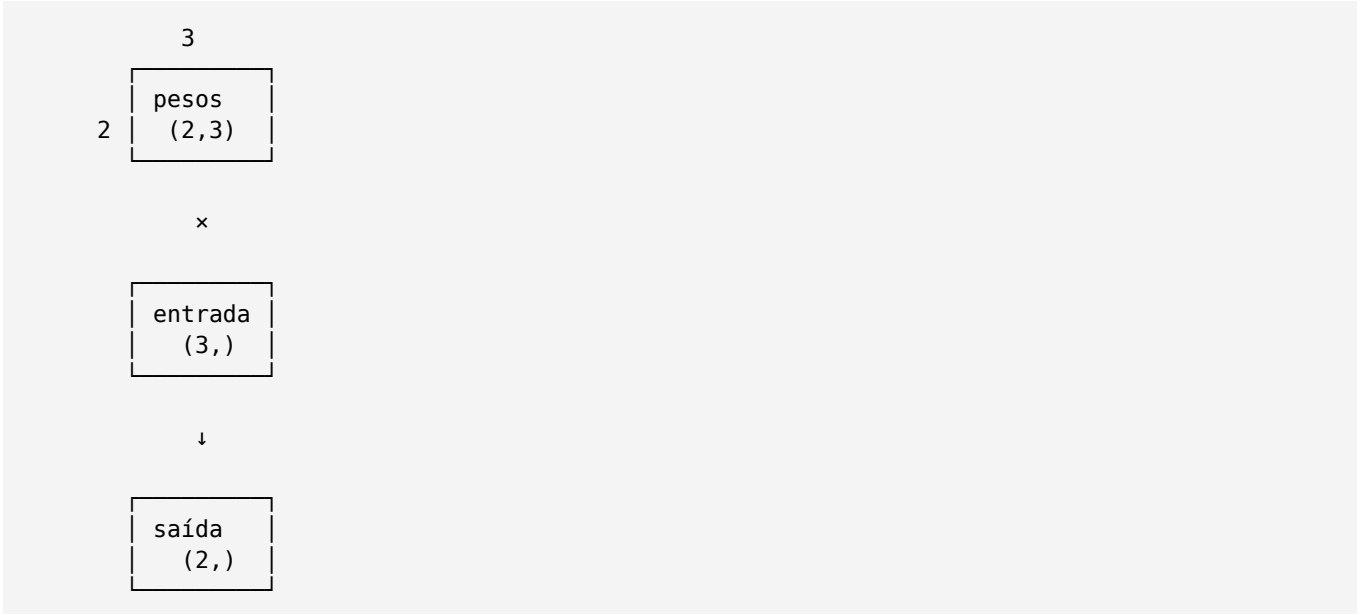
$z_1 = 0.2 + 0.1 = 0.3$
$z_2 = 0.3 + (-0.2) = 0.1$
z = Wx + b

Essa expressão calcula os valores anteriores à ativação para toda a camada. Ainda não aplica uma função de ativação.

Aula 5 - Compatibilidade de dimensões

5.10 Condição para multiplicar

O número de colunas de W deve coincidir com o número de elementos de x. Para W com formato `(2, 3)` e x com formato `(3,)`, o resultado tem formato `(2,)`:




```
(2 × 3) × (3) → (2)
   └─┘
  dimensões compatíveis
```

5.11 Experimento com erro de dimensão

Substitua temporariamente o vetor de entrada por um vetor com apenas dois valores:

```
entradas = np.array([1, 0])
```

A matriz continua com três pesos por neurônio, mas agora existem somente duas entradas. A operação `np.dot(pesos, entradas)` gera um erro de alinhamento de dimensões.

```
pesos    → (2, 3)
entradas → (2,)
```

Para investigar esse tipo de erro, confira os formatos dos objetos que participam da operação:

```
print(entradas.shape)
print(pesos.shape)
```

Exercício: execute o caso incompatível, leia a mensagem de erro e restaure `entradas = np.array([1, 0, 1])`.

Aula 5 - Programa completo

5.12 Prática: 05_numpy.py

```
import numpy as np


entradas = np.array([1, 0, 1])

pesos = np.array([
    [ 0.5, 0.2, -0.3],
    [-0.4, 0.8,  0.7]
])

biases = np.array([0.1, -0.2])


print("Formato das entradas:", entradas.shape)
print("Formato dos pesos:", pesos.shape)
print("Formato dos biases:", biases.shape)

z = np.dot(pesos, entradas) + biases

print("Resultado:", z)
```

Saída esperada, com possíveis pequenas diferenças de ponto flutuante:

```
Formato das entradas: (3,)
Formato dos pesos: (2, 3)
Formato dos biases: (2,)
Resultado: [0.3 0.1]
```

5.13 Equivalência entre as implementações

Forma	Cálculo
Explícita	Uma multiplicação escrita para cada par entrada-peso
Listas e for	Acumulação das multiplicações em um laço
NumPy	np.dot(pesos, entradas) + biases

As três formas calculam as mesmas duas somas ponderadas com bias. A organização matricial reduz a repetição de código e explicita a relação entre o número de entradas e o número de neurônios.

Verificação: os resultados das Aulas 4 e 5 devem coincidir: $z_1 \approx 0{,}3$ e $z_2 \approx 0{,}1$.

Aula 5 - Ligação com a classe Network

5.14 sizes e dimensões dos pesos

A lista `sizes` indica o número de unidades de cada camada. `Network([3, 2])` representa três entradas e dois neurônios na camada seguinte. São necessários 3 × 2 = 6 pesos, organizados em uma matriz 2 × 3.

```
rede = Network([3, 2])

# Matriz de pesos para essa ligação:
np.random.randn(2, 3)
```

No código de referência, `np.random.randn(y, x)` cria uma matriz com y linhas e x colunas. Aqui, y é a quantidade de neurônios da camada de destino; x é a quantidade de entradas vindas da camada anterior.

```
self.weights = [
    np.random.randn(y, x)
    for x, y in zip(sizes[:-1], sizes[1:])
]
```

Esses nomes locais x e y representam tamanhos de camadas; não representam, nessa linha, uma imagem e seu rótulo. O detalhamento de `zip` e dos recortes de listas não faz parte do conteúdo desenvolvido até esta aula.

5.15 Biases: atenção ao formato

```
self.biases = [
    np.random.randn(y, 1)
    for y in sizes[1:]
]
```

A classe cria biases com formato `(y, 1)`. Já os exemplos da Aula 5 usam um vetor 1D de formato `(y,)`. As representações precisam ser consistentes ao calcular `Wx + b`.

Convenção	Entrada x	Bias b	Resultado z
Arrays 1D da Aula 5	(3,)	(2,)	(2,)
Vetores-coluna	(3, 1)	(2, 1)	(2, 1)

Nota de estudo: misturar um resultado `(2,)` com um bias `(2, 1)` pode produzir uma matriz `(2, 2)` por expansão automática de dimensões (broadcasting). O formato do resultado deve ser conferido, além de seus valores.

Ponto de chegada

O percurso vai de pixels a vetores, de vetores à soma ponderada e de vários neurônios à operação `z = Wx + b`. A aplicação de ativações em múltiplas camadas e o aprendizado dos parâmetros são os próximos temas; ainda não foram desenvolvidos nas aulas reunidas nesta edição.